APPLE'S ON-DEVICE FOUNDATION MODEL, REBUILT: A COMPLETE FROM-WEIGHTS RECONSTRUCTION OF `AFMPLUS-V11.2-IFP` (AFM 3 CORE ADVANCED) AND A PORTABLE LLAMA.CPP RUNTIME

YU YAO-HSING

Abstract. We reconstruct Apple's current on-device foundation language model—asset `com.apple.fm.language.instruct_3b.base.generic_sparse`, display version `afmplus-v11.2-ifp`, the model Apple calls *AFM 3 Core Advanced*—entirely from the files Apple Intelligence installs on a Mac, and run it in llama.cpp on any operating system. The model is a 56-layer, 20.3-billion-parameter transformer with a 1536-wide residual stream, 16 query and 8 key/value heads, KV-cache sharing across its last 21 layers, an unusual post-normalised ("x/y") residual with four learned RMSNorm gains per layer, and an Instruction-Following-Pruned sparse feed-forward network: 23 always-on experts plus 23 of 348 routed experts per token, chosen by a separate router network evaluated once at layer 12. Every 2-bit weight is carried into GGUF *bit-exactly* (55 million spot-checked weights, zero differences); the tokenizer reproduces Apple's own token counts on 81/81 test texts. The two things the model files do not contain—the prompt template and the rule that picks experts—we read out of Apple's inference frameworks: the conversation is rendered as a system turn plus a separate developer turn, and the experts for a whole prompt are chosen from the last prompt token's hidden state and re-chosen every 32 positions. Both are confirmed by Apple's public token counter and by Apple's own runtime log. Against Apple's live model at temperature zero, the reconstruction's next-token choice agrees with Apple's on **97.0%** of answer tokens (95.3% on a harder 230-prompt set), and 17 of 40 free-running answers are identical to Apple's character for character. The last five points came late. llama.cpp applied the embedding scale twice (Apple applies none), Apple adds the layer-0 input bias only to generated tokens, and the Neural Engine's SiLU is a 33-knot piecewise-linear table that we now reproduce. Nothing in the model is trained or fitted. We also decode the 26 Apple Intelligence feature adapters (rank-48 LoRAs), Apple's 9M-parameter speculative-decoding draft model, with which speculation in llama.cpp is lossless, and the 358M-parameter image encoder, which matches Apple's own encoder run on the Neural Engine (cosine 0.96–0.99): through it the reconstruction predicts Apple's image captions 94.0% of the time teacher-forced, as well as with Apple's own encoder, and 5 of 13 free-running captions are identical to Apple's. Finally we decode Apple's 300M base model (`afmplus-v11.0-pico`) and the adapters built on it: with its default adapter it reaches a perplexity of 21–31 on plain English, Apple's safety guard, run through llama.cpp, answers `safe` for benign text and labels harmful text in its own category format, and through the 300M model's own image encoder its people counter is right on 9 of 10 photos.

For most of this project the assembled model predicted nothing. The cause was a single misaligned read: every normalisation gain was taken one element early. We found it with probes *calibrated* on a healthy Apple model—Xcode's on-device code model, whose MPSGraph package we dumped to readable MLIR, executed on the GPU from our own process and reproduced exactly in NumPy. The report documents the model's full structure, the storage formats, the debugging instruments and their calibration, the prompt template and expert schedule recovered from Apple's runtime, the llama.cpp implementation, a standalone converter, every negative result, and what still separates the port from 100%.

Contents

1. Results at a glance 2
2. Which model is this? 2
3. Where the model lives 2

Date: November 9, 2026.

4. Architecture 4
4.1. Dimensions 4
4.2. The layer: a post-normalised "x/y" residual 4
4.3. Attention 5
4.4. The Instruction-Following-Pruned feed-forward network 5
5. How the weights are stored 6
6. The norm-gain table, and the bug that hid the model 7
7. Validation instruments and their calibration 7
8. What Apple's runtime does: the prompt and the expert schedule 8
8.1. The prompt template 9
8.2. The expert schedule 10
9. The llama.cpp implementation 10
10. Apple Intelligence feature adapters 12
11. The speculative-decoding draft model 12
12. The image encoder 14
13. The 300M base model 16
14. Fidelity 18
15. The converter 19
16. What did not work 20
17. Toward 100% 21
18. Responsible use 24
Appendix A. GGUF layout 24

1. Results at a glance

Code: `https://github.com/yu314-coder/afm-ifp-teardown` (llama.cpp patch, converter, build workflow). No Apple model data is published; the converter rebuilds the GGUF from the assets on the user's own machine.

2. Which model is this?

Apple's third-generation foundation models (announced June 2026) comprise two on-device language models and several Private Cloud Compute models (Table 2). The macOS 27 SDK exposes the on-device model as `SystemLanguageModel` and the server models through `PrivateCloudComputeLanguageModel`. The assets on the device contain exactly one foundation language backbone, `afmplus-v11.2-ifp`; the server models are present only as adapter metadata (`afmplus-v8.0-150b`, `afmplus-v9.0-3b-pcc`).
So the reconstructed model is *AFM 3 Core Advanced*, not a "Pro" model: the Pro model runs only on Apple's servers and is not on the device.

3. Where the model lives

Apple Intelligence downloads its models as MobileAssets under `/System/Library/AssetsV2/com_apple_MobileAsset_UAF_FM_GenerativeModels`. The model files sit in an APFS cryptex that the system grafts into the asset's `.AssetData` directory.
Reading Apple's graphs. A private method on `MPSGraphExecutable`, `-getIR`, returns a loaded package's graph as MLIR text. On AFM it yields only placement operations and 108 calls into the ANE program, but with exact entry signatures (for example, the two 64×128 RoPE tables arrive as runtime inputs). On Xcode's code-completion model (CodeLM v3, a GPU model built

Model	`afmplus-v11.2-ifp` = AFM 3 Core Advanced (on-device, sparse IFP)
Size	20.30 B parameters stored; 1–4 B active per prompt (ifp1/2/3 budgets)
Layers	56 = 12 dense + 23 sparse (own K/V) + 21 sparse (reuse layer-34 K/V)
Width	$d = 1536$; 16 Q / 8 KV heads × 128; dense FFN 3072; expert width 256
Weights	2-bit palette $\{-1.5, -0.5, 0.5, 1.5\}$ × fp16 row scale; int4 embedding
GGUF	6.31 GiB, Q2_K (bit-exact) + Q4_0 (exact); custom arch `afm`
Fidelity	97.0% teacher-forced top-1 vs Apple (40 prompts, 1127 tokens), 95.3% on 230 prompts; 17/40 answers identical
Runtime rules	`afm_cider_v2` template and IFP expert schedule, decoded from Apple's frameworks
Tokenizer	matches Apple's `SystemLanguageModel.tokenCount` on 81/81 texts
Runtime	llama.cpp fork: Metal, CUDA, Vulkan, CPU; 150 tok/s prompt, 12–21 tok/s generation (M4)
Converter	`afm2gguf.py`: 2–3 min, reads only the local assets, self-validating
Draft model	Apple's speculative-decoding draft (`v11-ifp-9m`, 12 layers, $d = 256$) decoded: 43% next-token agreement with Apple; exact `afm-draft` GGUF; speculation lossless (40/40) but not faster in llama.cpp (Section 11)
Image encoder	24-layer windowed ViT + 3 × 3 merger (358 M) decoded; matches Apple's own encoder (cosine 0.987 per token on constant inputs, 0.95–0.97 on images); captions 94.0% teacher-forced (Apple's own encoder: 92.9%), 5/13 identical; llama.cpp projector `afm` (Section 12)
300M base	`afmplus-v11.0-pico`, the base of Apple's 300M classifiers (24 layers, $d = 1024$) decoded: perplexity 21–31 on plain English with its default adapter, 78% induction copying, coherent chat answers; `afm-draft` GGUF (Section 13)
Safety guard	Apple's 300M safety classifier and 11 other task adapters decoded as LoRAs; the guard answers `safe` for benign text and labels harmful text in its own `category,action/mention` format (Section 13)
300M images	the 300M model's own image encoder (same structure as the 3B's, output 1024) decoded; its people counter is right on 9 of 10 photos with 0–2 people, and its image safety classifiers pass every benign image (Section 13)
Root cause of the long failure	norm-gain table read 2 bytes (one fp16) early

Table 1. Headline numbers. All measurements in this report were taken on an M4 Mac running macOS 27.

Name	Where	Size	Architecture	On this Mac?
AFM 3 Core	device	3 B	dense	no separate asset found
AFM 3 Core Advanced	device	**20 B**, 1–4 B active	**sparse, IFP**	**yes: `afmplus-v11.2-ifp`**
AFM 3 Cloud	PCC	n/a	n/a	metadata only
AFM 3 Cloud Pro	PCC (GPUs)	n/a	n/a	no

Table 2. Apple's 2026 lineup as described in public WWDC 2026 coverage (secondary sources; Apple's own documentation does not list sizes). The match for v11.2 rests on our measurements: llama.cpp counts 20.30 B parameters in the converted model; its feed-forward network is Instruction-Following-Pruned; and the Foundation Models API ships three compute budgets (`fm_api_generic_1b/2b/3b`) that select 5, 14 or 23 routed experts—the "1–4 B active per prompt".

with the same modelling library, *tamm*) it yields the complete source-level graph. CodeLM became our healthy reference model (Section 7).

File	What it holds
`binary_0.hwx` (694 MB) in `main-h16g-delegates/MPSGraph/`	The compiled ANE program: 108 procedures (prompt / extend / dense-only / sparse-only / readout, per sequence length and IFP budget). Segment `__KERN_3` holds every attention and dense-FFN weight; `__TEXT,__const` holds the norm gains; the task stream holds the int8 K/V calibration.
`main-h16g.odix` (263 MB)	The orchestration program (ODIE IR). Holds the int4 token embedding (text, audio) and its per-token fp16 scales, and the RoPE tables.
`ifp/ifp_rasterized_weights.bin` (4.9 GB)	All 44×380 experts: 2-bit codes, gate/up and down row scales.
`ifp/project_experts.mlasset`	The IFP router network (fp16).
`ifp/config.json`	IFP budgets: ifp1/2/3 = 5/14/23 active experts; `expert_selection_frequency` = 32.
`tokenizer` (separate asset)	328 192-entry SentencePiece-BPE vocabulary in a flat Apple format (262 144 text tokens + audio/voice tokens).

Table 3. Files used by the reconstruction.

4. Architecture

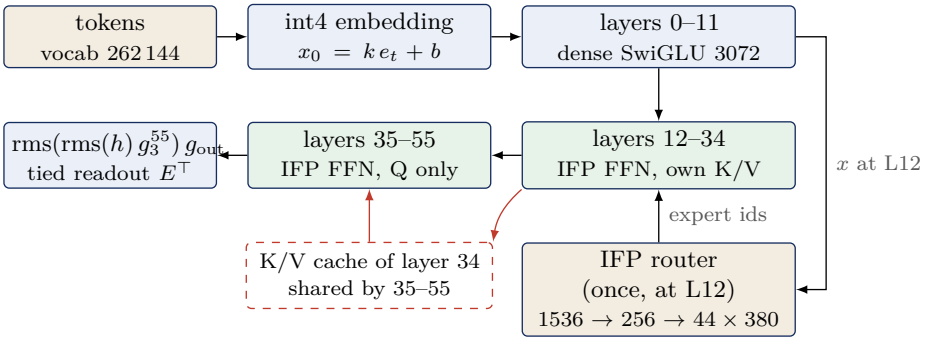


Figure 1. The whole model. The router reads the normalised input of layer 12 and emits routing logits for all 44 sparse layers at once. Layers 35–55 compute queries only and attend to the K/V cache written by layer 34.

4.1. Dimensions.

4.2. The layer: a post-normalised "x/y" residual.

Each layer carries four learned RMSNorm gains $g_0, \ldots, g_3 \in \mathbb{R}^{1536}$. Unlike a pre-norm transformer, both residual additions add *normalised* values:

$$
\begin{aligned}
x_L &= \mathrm{rms}(h_{L-1}) \odot g_3^{L-1} \qquad \text{(layer 0: } x_0 = k\,e_t + b,\ \text{no norm)} \\
h'_L &= x_L + \mathrm{rms}(\mathrm{Attn}_L(x_L)) \odot g_0^L \\
y_L &= \mathrm{rms}(h'_L) \odot g_1^L \\
h_L &= y_L + \mathrm{rms}(\mathrm{FFN}_L(y_L)) \odot g_2^L
\end{aligned}
$$

with $\mathrm{rms}(v) = v/\sqrt{\mathrm{mean}(v^2) + \varepsilon}$, gains used literally (no $1 + g$), and a final output $\mathrm{rms}(\mathrm{rms}(h_{55}) \odot g_3^{55}) \odot g_{\text{out}}$ multiplied by the tied embedding. The residual wiring was read from the DMA pattern

Quantity	Value	Evidence
layers	56 (12 dense, 23 sparse own-KV, 21 sparse KV-reuse)	chunk-set plan; task stream
hidden / head dim	1536 / 128	conv shapes
heads	16 query, 8 key/value; query head h uses KV head $\lfloor h/2 \rfloor$	attention task addressing; same-token probe
dense FFN	SwiGLU, width 3072 (layers 0–11)	conv shapes; SiLU lookup table on the gate
experts	380 per sparse layer, width 256; 0–22 always on; 32–379 routable	router mask; static kernels = experts 0–22
RoPE	NEOX half-split, $\theta = 5 \times 10^5$, 128 dims	odix cos table (row 0 = 1, halves equal)
norm ε	1.0014×10^{-5}	ANE register; identical in CodeLM
attention scale	$1/\sqrt{128}$	post-scale = $(1/s_K)/\sqrt{128}$ in the hardware; CodeLM `sdpa`
vocabulary	262 144 text tokens (Gemma-3 family layout), BOS 2, end-of-turn 106	tokenizer asset
context	8192	asset metadata

Table 4. Architecture parameters and where each was read.

of the ANE program: the normalised attention input is written to DRAM scratch and read back by the attention residual add; the FFN residual add reads the normalised FFN input. Only this wiring reproduces Apple's outputs (standard residual: 0% top-1).

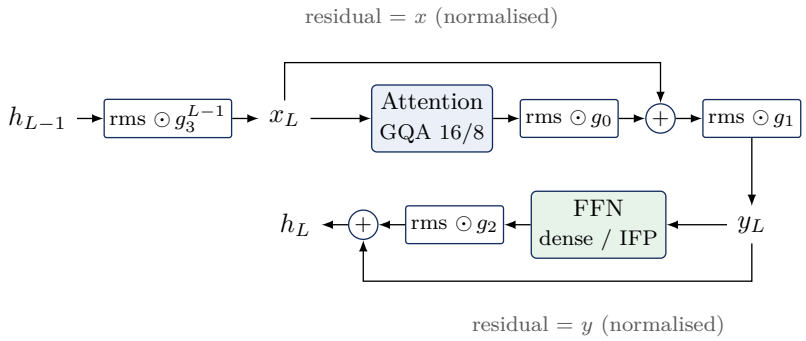


Figure 2. One layer. Both residual additions add normalised streams; g_0 and g_2 are zero-centred (about 30% negative entries), g_1 and g_3 small and positive. Zero-centred literal gains are normal in Apple's modelling library: a healthy Apple model (CodeLM) shows the same pattern.

4.3. **Attention.** Attention is standard grouped-query attention with two Apple-specific details: the query/key normalisation sits after RoPE, and the key/value cache is int8 (Figure 3).

4.4. **The Instruction-Following-Pruned feed-forward network.** Sparse layers add two SwiGLU blocks without any gating weights:

$$\mathrm{FFN}(y) = \underbrace{\sum_{e=0}^{22} \mathrm{SwiGLU}_e(y)}_{\text{always on}} + \underbrace{\sum_{e \in \mathcal{S}(y)} \mathrm{SwiGLU}_e(y)}_{|\mathcal{S}|=23 \text{ routed}}, \qquad \mathrm{SwiGLU}_e(y) = W_e^{\mathrm{down}}\big(\mathrm{SiLU}(W_e^{\mathrm{gate}} y) \odot W_e^{\mathrm{up}} y\big),$$

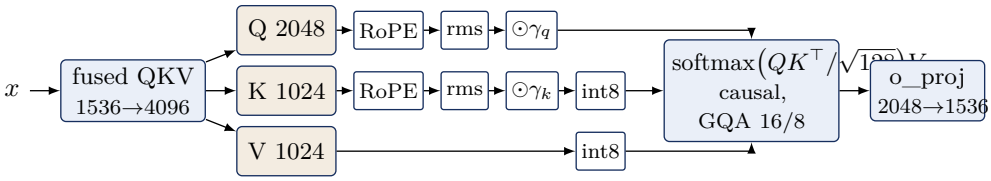


Figure 3. Attention as executed on the ANE. RoPE comes before the per-head RMS norm and its 128-entry gain (the same order appears in CodeLM's source graph). Keys and values are stored in an int8 cache with per-layer calibrated scales; layer 34 has no key gain. Layers 35–55 compute only Q and read layer 34's cache.

each expert being 256 wide. The ANE program runs the always-on experts from static kernels and the routed experts from a mutable kernel patched at runtime from `ifp_rasterized_weights.bin`. Both halves matter: the always-on experts alone reach 62.9% top-1, the routed experts alone 1.4%, both together 87.5% (measured with the earlier template).

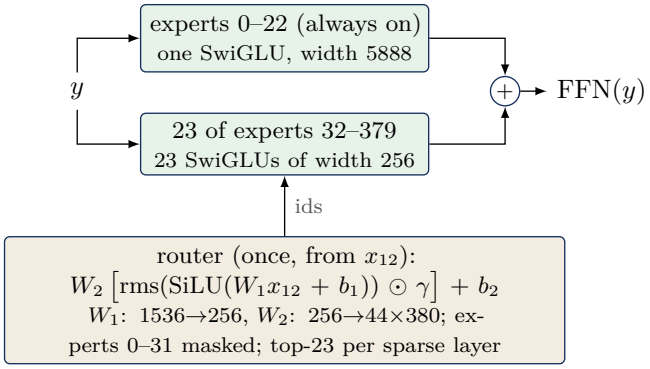


Figure 4. The IFP feed-forward network of a sparse layer and the router that feeds it. The sum is unweighted: the router only selects.

When experts are chosen. Apple's runtime does not route per token. It runs the dense layers over the whole prompt, takes the layer-12 input of the *last* prompt token, runs the router once, and uses that expert set for every prompt token and for generation; it re-picks from the current token only at absolute positions that are multiples of 32. Section 8 shows how this was read out of Apple's inference framework.

5. How the weights are stored

Exact GGUF encoding. llama.cpp's Q2_K stores, per 256 weights, sixteen 4-bit sub-block scales sc and mins m, a super-scale d and super-min $d_{\min}$, and evaluates $w = d\,sc\,c - d_{\min}\,m$. Choosing $sc = 2$, $m = 3$ and $d = d_{\min} = s/2$ gives

$$w = \tfrac{s}{2} \cdot 2c - \tfrac{s}{2} \cdot 3 = s\,(c - 1.5),$$

and halving an fp16 number is exact (barring underflow, which never occurs here). The int4 embedding ($w = s_t\,q$, $q \in [-7, 7]$, per-token fp16 scale s_t) maps exactly onto Q4_0 with $d = s_t$ and nibble $q + 8$. Dequantising the GGUF with gguf-py reproduces Apple's weights with maximum absolute difference 0.

Layout facts and how they were ground-truthed. We compiled known weights through Apple's own ANE compiler (`mil_to_hwx -a h16`) and decoded the result: the output axis is the bank-cyclic stripe above, the input axis is natural, the codebook and row scales are as in Figure 5, and constants (gains, biases) are stored in natural order. A calibrated copy-circuit probe (Section 7) independently confirms the stripe, the head order and the V/O pairing: every alternative layout scores at chance.

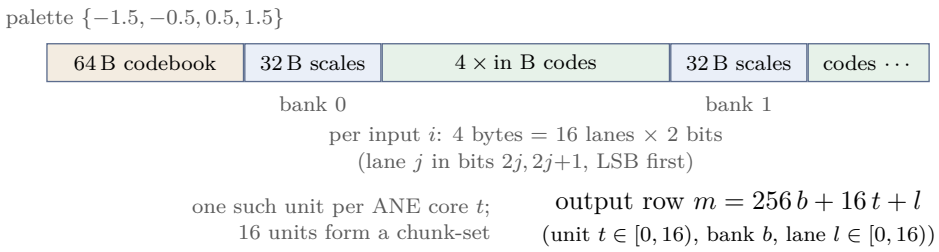


Figure 5. The 2-bit weight unit. A projection is split across the 16 ANE units bank-cyclically; a chunk with more banks holds the lower bank slots. The value of a weight is $w = s_m\,(c - 1.5)$ with $c \in \{0, 1, 2, 3\}$ and s_m the fp16 row scale. Experts in `ifp_rasterized_weights.bin` use the same codes with a simpler order: code $p \mapsto$ (row $16\lfloor p/(16\,\text{in})\rfloor + p \bmod 16$, input $\lfloor p/16\rfloor \bmod \text{in}$).

6. The norm-gain table, and the bug that hid the model

All gains live in one contiguous fp16 table inside `__TEXT,__const` of `binary_0.hwx`:

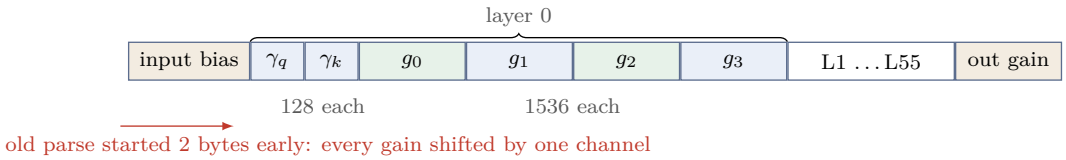


Figure 6. The gain table. Layers 0–33 carry γ_q and γ_k; layers 34–55 only γ_q. Every vector length is a multiple of 64 bytes, so a correct start is 64-byte aligned.

Finding 1. The earlier parse located the table by statistical signatures and accepted a start two bytes early. Every RMSNorm therefore multiplied channel r by the gain of channel $r-1$. Because g_0 and g_2 are zero-centred, this flipped the sign of about a third of every residual write at every layer, and the assembled model predicted nothing (0% top-1, no induction).

How it was found. Nothing looked wrong in isolation: each tensor was correctly laid out and correctly paired. The breakthrough was calibrating probes on a healthy Apple model and then comparing *reader–reader* with *reader–writer* relations. In AFM, query–key and gate–up relations were as healthy as in CodeLM, but value–output (copy heads) and up–down (FFN neuron coupling) relations were weak. In a trained network, the sign of the coupling between up-row i and down-column i agrees with the sign of the gain that scales the write. In AFM the *number* of negatively coupled channels equalled the number of negative entries of g_2 in every layer—while their *positions* disagreed (Figure 7, left). Matching each channel's 11-layer sign signature against the table gave a constant offset of one element. Read aligned, the agreement rises to 98–99.9% (right) and the model works: 66% top-1 against Apple's outputs on the first attempt, rising to 87.5% with the llama.cpp port and exact tokenization.

7. Validation instruments and their calibration

The oracle. `afm -t 0` (a small Swift wrapper around the Foundation Models API) produces Apple's greedy answers; at temperature zero, Apple's top-1 token at every answer position is the next token, so teacher-forced top-1 agreement is an absolute measure. We use 40 prompts (1127 answer tokens). The API exposes no probabilities.

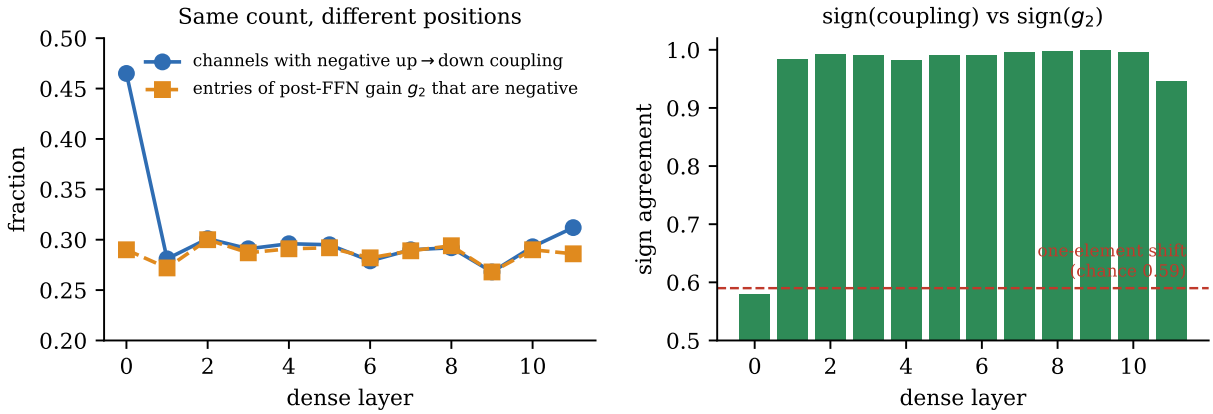


Figure 7. Left: per layer, the fraction of residual channels with negative up→down coupling (from the weights) and the fraction of negative entries in the decoded post-FFN gain g_2—equal counts. Right: per-channel sign agreement after reading the table aligned; a one-element shift gives chance level. Layer 0 is the exception because its input is the raw embedding.

A healthy reference: CodeLM v3. Xcode's on-device code model ships as a GPU MPSGraph package. Its MLIR (via `-getIR`) is a complete, readable tamm transformer—pre-norm, standard residual, RoPE then QK-norm, SwiGLU, KV reuse from layer 34, int4 weights with group-32 scales—with the same backbone shape as Apple's other 3B-class models (56 layers, 35 KV-owning, 2048 wide, 16/2 heads). We ran it on the GPU from our own process through `MPSGraphExecutable` (100% induction accuracy) and reproduced its logits in NumPy exactly (correlation 1.00000, 100% argmax agreement). Its tokenizer and embeddings are unrelated to v7-3b's, so it cannot donate weights—but it calibrates instruments.

What the calibration showed.

- *Zero-centred literal gains are normal* in tamm (CodeLM's pre-attention gain: mean +0.003, 46% negative).
- *The embedding–weight "alignment ratio" used for months is not diagnostic*: a healthy model scores the same (1.0–1.4).
- *The OV copy probe is diagnostic*: CodeLM has several heads that map a token to itself at rank 0–16, even after naive 2-bit rounding; decoded AFM had none below rank 1000 (Figure 8). Every alternative V/O layout scored at chance, which ruled out layout errors and pointed at the gains.
- *The same-token QK probe* found AFM's query–key circuits healthy (rank-0 heads) and confirmed the head pairing $h \mapsto \lfloor h/2 \rfloor$.

Apple's int8 calibration as a ground truth. The ANE program stores per-layer scales for its int8 K/V cache. Apple chose them from real activations, so they bound the true magnitudes. With the corrected gains, the reconstruction's values stay inside Apple's ranges, and its keys stay inside up to the 99.9th percentile; only the most extreme 0.1% of keys would clip (Figure 9). This is an independent check of the internal scales. (Figure 9 was made with the v1.0 forward; with the correct layer-0 input the layer-0 values use only part of Apple's range, so Apple's layer-0 scale is not a tight bound.)

8. What Apple's runtime does: the prompt and the expert schedule

Two things the model files do not contain decide what the model sees and which experts it uses: the prompt template (the metadata only names it, `afm_cider_v2`) and the rule for choosing experts.

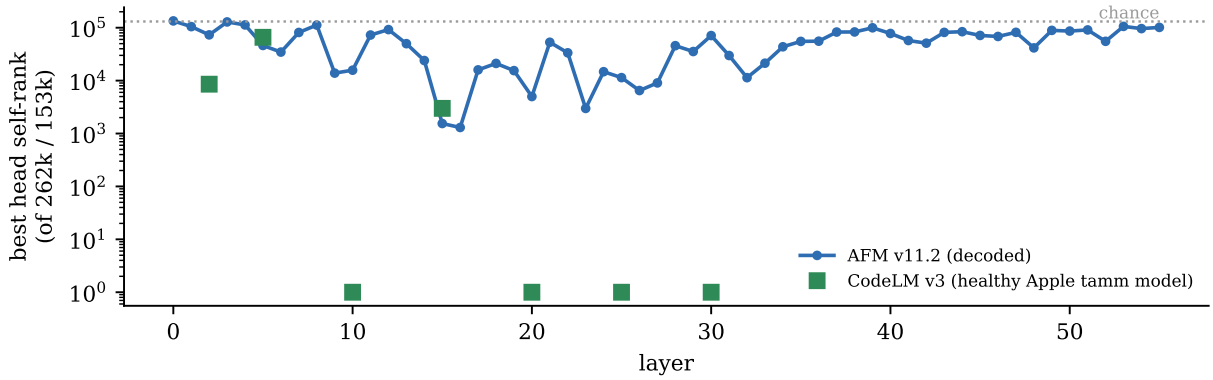


Figure 8. Copy-circuit probe: for each layer, the best attention head's median rank of the input token in $E\,W_O W_V\,e_t$. Healthy CodeLM has rank-0 heads; AFM's raw $W_O W_V$ peaks at weak copying because in AFM the write passes through the signed gain g_0, whose positions were scrambled by the table shift.

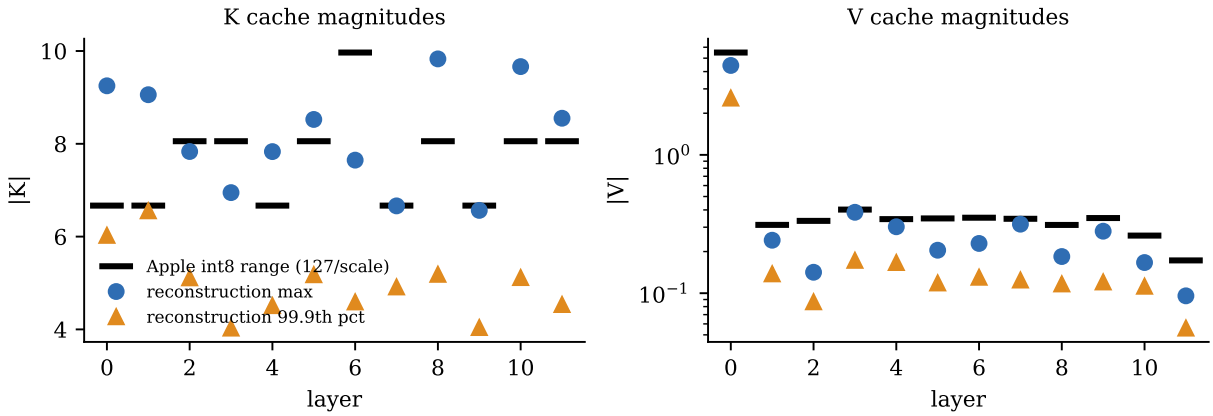


Figure 9. Reconstructed key and value magnitudes (dense layers, one prompt) against Apple's calibrated int8 ranges. Layer 0's values are large because its input is the un-normalised embedding.

Both live in code, in two private frameworks of the dyld shared cache: `TokenGenerationCore` renders and tokenizes prompts, `TokenGenerationInference` drives the model. `dyld_info -disassemble` prints both with Swift symbol names. One detail matters for reading them: Swift stores string literals of up to 15 bytes inside the instruction stream (`mov`/`movk` immediates), so `<start_of_turn>` and friends never show up in a strings dump.

8.1. **The prompt template.** `PromptPreprocessingTemplateAfmCiderV2Convention` takes its special-token table from the older Cider v1 convention and overrides only audio and reasoning entries. Decoding the immediates gives turn start `<start_of_turn>`, turn end `<end_of_turn>`, and roles `system\n`, `developer\n`, `user\n`, `model\n`; the "thought budget: none" and "developer static instructions" entries are empty. Its `cacheableSystemPrefix` concatenates turnStart, the system role with Apple's default instructions, the thought budget, turnEnd, a newline, then turnStart and the developer role. A FoundationModels conversation therefore renders as

`<bos><start_of_turn>system\n`*default*`<end_of_turn>\n`
`<start_of_turn>developer\n`*instructions*`\n<end_of_turn>\n`

<start_of_turn>user\n*prompt*<end_of_turn>\n<start_of_turn>model\n

where *default* is the systemInstructions.default text of the adapter metadata ("A conversation between a user and a helpful assistant. You are a foundation model developed by Apple. . . . ") and the developer turn holds the app's instructions. The public API verifies every detail: SystemLanguageModel.tokenCount(for:) accepts transcript entries and returns the length of the rendered prompt, which equals the length the inference service logs ("asked to run 71 tokens") for the same conversation. Table 5 compares it with our rendering. Our earlier template folded everything into one system turn and omitted the instructions our reference CLI passes; with the exact template, teacher-forced agreement rose from 87.5% to 90.5% and the first answer token from 30 to 38 of 40.

Transcript (I = instructions, P = prompt, R = response)	Apple tokenCount	our template
P only	16	16
I (empty), P	62	62
I only	55	55
I, P	71	71
I, P, R, P	86	86

Table 5. Prompt lengths. A session without instructions still carries the system turn and an empty developer turn (62 tokens, seen in the live log); each instruction segment ends with a newline; earlier answers are closed with <end_of_turn>\n.

8.2. **The expert schedule.** The inference log of a single request already shows the shape of the algorithm: a "prompt expert compute function" (lora_48_ifp3_r48_prompt_opt_dense_only_256_64, the dense layers only) runs over all prompt tokens, then "Picking IFP experts at position=70 isPromptPick=true" for a 71-token prompt, then the sparse layers run over the *whole* prompt again with the picked experts. The dense-only function returns a 1536×64 fp16 tensor per 64-token chunk, one column per token. In TokenGenerationInference, computePromptExpertState builds the router input with IFPDenseLayerOutput.makeExpertsTensor(tokenIndex:), whose loop copies the 1536 values of a single column into a 1536×1 tensor—no mean, no accumulation. The router (project_experts, the network of Figure 4) then yields the top-23 experts per sparse layer. During generation, repickExpertsAtBoundary calls the same picker, and IFPExpertPicker rejects any intermediate pick whose position is not a multiple of expert_selection_frequency (an sdiv/msub remainder test against 32). With prompt length P, token t therefore uses the experts chosen from x_{12} at

$$s(t) = \max(P - 1,\ 32\lfloor t/32 \rfloor),$$

so every prompt token uses the last prompt token's choice. Averaging the prompt instead is clearly wrong (79.3% teacher-forced), while the single-token rule beats per-token routing (Section 14).
An outside check of the router. Each pick also logs CacheExpertExpertCache: ... replaced=n, the number of (layer, expert) choices that changed. We ran six long generations on Apple's model, collected its 47 re-picks, replayed the same token sequences through our model and counted how many of our choices change at the same positions. With the single-token rule on the layer-12 attention input the two series agree closely (Figure 10: mean 26.7 versus 27.1 experts, correlation 0.84); prefix means correlate at 0.21–0.53 and the FFN-input variant changes three times too many. This validates the router input, the router network and the rule independently of any answer text.

9. The llama.cpp implementation

The fork adds one architecture, afm, in src/models/afm.cpp (about 180 lines), plus three supporting changes.

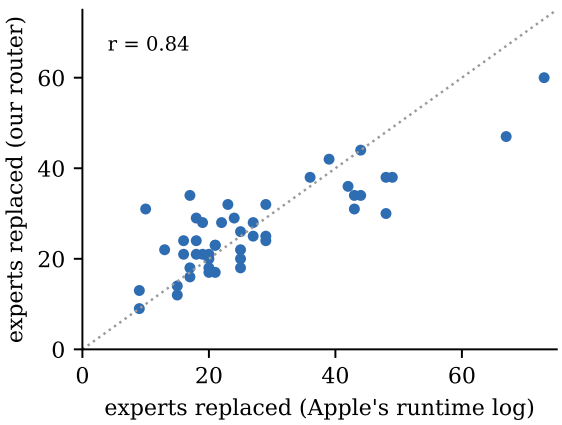


Figure 10. Experts replaced at each re-pick during generation: Apple's runtime log versus our router replaying the same tokens (47 re-picks in six generations).

- **Graph.** Layer 0 takes the dequantised embedding row as is (no multiplier) and, for generated tokens other than re-pick tokens, adds the input bias (Section 14); each later layer normalises the previous output with g_3; queries and keys use `rms_norm`, NEOX RoPE, then a gain multiply; the residuals follow Figure 2. Sparse layers run the always-on experts as a "shared expert" SwiGLU of width 5888 and the routed experts with `mul_mat_id`, summed by one row reduction with no weights. The router network lives on layer 12 and its logits are sliced per layer. The output applies the double norm and the tied embedding.
- **KV reuse.** Layers 35–55 call attention with null K/V, and the cache maps them to layer 34. This needed a two-line fix in the non-sliding-window attention path, which assumed K/V were always present.
- **Metal kernels.** Metal's batched expert matmul is instantiated for specific experts-per-token counts; we added 14 and 23 (ifp2, ifp3). CUDA uses a generic path for such counts, and Vulkan supports Q2_K expert matmuls, so no change is needed there.
- **Apple's expert schedule.** The router's logits for each sequence persist in a small device buffer (64 sequence slots) owned by the model. A custom graph input computes, per micro-batch on the host, which logits every token uses: a run of several tokens of one sequence is a prompt chunk and takes its last token's logits, which are stored; a single decode token re-picks only when its position is a multiple of 32 and otherwise reads the stored choice. Because the indices are data rather than graph structure, llama.cpp's graph reuse and multi-sequence batching stay correct. Speculative verify batches are handled as single decode tokens, with two position-tagged picks per sequence so that rejected draft tokens leave no trace (Section 11). `AFM_ROUTE_TOKEN=1` restores per-token routing.
- **The embedding-scale pitfall.** llama.cpp's generic `build_inp_embd()` multiplies by the GGUF's `embedding_scale` (a Granite feature). The first release also multiplied in the AFM graph, so a GGUF carrying our fitted 10 ran at 100. The loader now ignores the key, so old GGUFs run correctly.
- **The Neural Engine SiLU.** The activation is the ANE's piecewise-linear table, built from standard ggml operations: a clamp, a magic-constant floor, two 38-entry `get_rows` tables and a relu for the right tail, then rounded to fp16. It costs about 45% of prompt speed and 20% of generation speed on Metal; `AFM_EXACT_SILU=1` switches back to the exact function.
- **Router on the CPU.** The router's top-23 margins (median 0.003) are close to the error of Metal's tiled matmul ($8 \cdot 10^{-5}$ against $2 \cdot 10^{-7}$ on the CPU; 0.002 expert swaps per layer-pick). Its weights are therefore placed on the CPU, which makes picks identical across backends.

The routing state and SiLU tables stay on the layer's device. Agreement is unchanged within noise.

- **int8 KV emulation.** The GGUF carries Apple's per-layer K/V scales; keys and values are rounded and saturated to int8 before caching. Rounding uses the fp32 magic constant $1.5 \cdot 2^{23}$, because `ggml_round` misbehaved on Metal.

On an M4 the model processes prompts at about 150 tokens/s and generates 12–21 tokens/s. The patch applies to upstream llama.cpp (commit `41ea261`); the repository's workflow builds Windows (CPU, Vulkan, CUDA), Linux (CPU, Vulkan) and macOS (Metal) binaries from it.

10. Apple Intelligence feature adapters

Apple Intelligence features ship as rank-48 LoRA adapters for this backbone: 26 on our Mac, covering summaries, proofreading, mail replies, smart replies, translation, message actions, reminders, Image Playground suggestions, Photos memories, dictation and others. Each asset holds `lora_i`N`_r48_48_constant_data.bin`, where N is the IFP level the feature runs at. Its metadata holds Apple's prompts for the feature (system instructions and content templates, in the `afm_cider_v2` format) and the same backbone signature as the base model. All of them carry the empty-input SHA-256 as their "adapter signature", so that field says nothing about whether an adapter is trained. The FM API's own adapter is untrained (B RMS $3 \cdot 10^{-5}$), while the feature adapters are trained (B RMS $2 \cdot 10^{-3}$ to $3 \cdot 10^{-2}$).

Layout. A file holds 218 (A, B) pairs as fp16, in segment order: dense (o, q, k, v, `linear_0`, `linear_1`, down) $\times 12$, then kv-reuse (o, q) $\times 21$, then standard sparse (o, q, k, v) $\times 23$. The pairs account for every byte. We chose the orientation, row order, gate/up assignment and scale with a likelihood test. For eight proofreading pairs of our own, the cost of the correct output under the proofreading adapter at ifp2 is 0.016 nats per token with A as $[48, \text{in}]$, B as $[\text{out}, 48]$, natural row order, `linear_0` = up and scale 1. It is 0.119 with no adapter, and at least 0.036 with any of the other 15 layouts. Scales 0.75 and 1.25 give 0.050 and 0.330.

IFP levels. ifp1 and ifp2 use the first 5 and 14 always-on experts plus as many routed ones. In llama.cpp these are views of the shared FFN; at 256-aligned offsets they are whole Q2_K blocks. The level is read from the active adapter (`adapter.afm.ifp`). With Apple's prompts, the proofreading, Catch-Up summary and mail-reply adapters produce the expected outputs, and the proofreading adapter fixes tense errors the base model leaves in.

11. The speculative-decoding draft model

Apple pairs the model with a small draft model for speculative decoding: asset `fm_api_generic.draft`, `model_config v11-ifp-9m`. Its metadata asks for 7 draft tokens per step, exact matching (`draft_soft_match_tolerance` 0) and no adaptive length. The asset has no weights file: everything is inside its Neural Engine program (`binary_0.hwx`, 22 procedures: 3 readouts of 8/16/64 tokens and 19 `extend` variants) and its `program.odix`.

Reading the dataflow. The 132 weight convolutions (11 per layer) are the tasks with Neural Engine opcode `0x46` and a kernel offset. Their roles follow from the surrounding tasks. In layers 1–11 the first 128-channel projection is written straight to the cache by a tile-destination DMA, so it is V. The second passes through RoPE, a 32-wide RMS reduction and a gain multiply before its cache write, so it is K. Layer 0 orders them q, k, v. The attention score scale $1/\sqrt{32}$ sits in the QK matmul's output-scale register, and the softmax is an exp2 with a $\log_2 e$ prescale. The 768-wide FFN hidden is laid out by L2 address: the three up and three gate tiles are written to descending addresses, so the hidden is $[\text{tile}_2 \mid \text{tile}_1 \mid \text{tile}_0]$. In the last layer the compiler re-banks these buffers and leaves one destination register implicit, so we use the conv order there. Norm gains come from `__TEXT,__const`, one 0x4000-byte block per procedure, addressed by the tile-source offsets. In layer 2 the query gain multiply reuses the tile the key gain loaded; the compiler had merged the two. The

Shape	12 layers, $d = 256$, 8 Q / 4 KV heads $\times$ 32, SwiGLU FFN 768; 9.4 M parameters plus the int4 embeddings
Layer	pre-norm: $h = \mathrm{rms}(x)g_1$; $q, k =$ RoPE (half-split, base $5 \cdot 10^5$) *then* per-head RMSNorm and gain; $x \mathrel{+}= W_o\,\mathrm{attn}$; $x \mathrel{+}= \mathrm{FFN}(\mathrm{rms}(x)g_2)$
Weights	4-bit palettes: each projection is 16 units (one per ANE core), each a sorted 16-entry fp16 codebook, then input-major nibbles; no row scales. Gate tiles carry the 128-byte ANE SiLU table
Vocabulary	100,000 of the 262,144 tokens (a sorted id list in `program.odix`); 95.9% of Apple's answer tokens are in it
Embeddings	int4 codes for the 162,144 other tokens, then the 100,000 draft tokens, in the base model's 8-token interleave, times one per-dimension fp16 scale s; no per-token scale
Readout	tied: the readout procedure's constant tile is $[\,0 \mid s\,]$, i.e. $h \cdot s + 0$, then the int4 matmul over the draft vocabulary

Table 6. The draft model as recovered from its Neural Engine program.

query and key gains of a layer agree up to sign flips inside rotation pairs and a few ulps, so the norm cannot be moved before RoPE. The final norm computes $\mathrm{rsqrt}(\mathrm{mean}((0.1x)^2) + 1.19 \cdot 10^{-7}) \cdot 0.1$, an fp16 overflow guard equivalent to $\epsilon = 1.19 \cdot 10^{-5}$.

Validation. A NumPy forward of this description predicts Apple's own next answer token (from the 3B model, teacher-forced) 42.95% of the time on the 40 prompts (1127 tokens) and 37.7% on the 230 (6572 tokens). Every alternative we tried collapses: cyclic instead of unit-blocked output rows 0.0%, reversed RoPE direction 16.7%, query head $h \to$ KV head $h \bmod 4$ 6.7%, K and V swapped 1.5%, FFN tiles in conv order 2.3%, raw codes without s 0.3%. The input scale is a sharp optimum at 1 (0.25: 27.6%, 4: 36.5%). The ANE SiLU table changes nothing (42.95% either way). Simulating Apple's speculation (7 tokens, exact match), the draft's proposals are accepted 0.61 tokens per step on the 230 prompts, so each target call yields 1.61 tokens.

In llama.cpp. `afm2gguf.py --draft` writes the draft as architecture `afm-draft` (61 MiB): the codes as Q4_0 with $d = 1$ (exact), s as `token_embd_scale` and `output_scale`, F16 projections (palette values are fp16), and an `output.bias` of $-\infty$ outside the draft vocabulary. The bias matters: the best draft logit is negative at about 1% of positions, so zero rows would sometimes win. On Metal, its logits match the NumPy reference to 0.03.

Speculative decoding exposed a problem in the target graph. A verify batch [last token, 7 draft tokens] is several tokens of one sequence, which the `afm` graph had treated as a prompt chunk: one shared expert pick, persisted, and no input bias. Verify batches are now recognised (every token requests logits and the run does not start at position 0) and processed exactly as single decode tokens: input bias except at re-pick positions, and routing on the latest re-pick token at or before each token. Draft tokens can be rejected after they have re-picked, so the routing state now keeps two picks per sequence, each tagged with its source position. A pick from a position the batch starts at or after belongs to rolled-back tokens and is ignored. With these changes, 8-token verify batches reproduce one-token-at-a-time decoding to $2 \cdot 10^{-5}$ in the logits.

Lossless, but not faster in llama.cpp. With greedy sampling, 7 draft tokens and no confidence cutoff (Apple's settings), `llama-server` with the draft returns exactly the plain answers on all 40 prompts, on Metal and on the CPU, and the plain answers are unchanged by the patch. About 11% of drafted tokens are accepted, as the simulation predicts. Speculation is nevertheless slower in llama.cpp. Measured back to back, it was 2.1× slower on Metal and 3.5× slower on the CPU (1.8× with 3 draft tokens). A verify batch costs several single decodes here, because the 2-bit expert matmuls are compute-bound and every draft step has a fixed per-call cost. Apple's Neural Engine runs fixed-shape 8-token `extend` programs, so a verify step costs about as much as one

token, and 0.6 extra tokens per step is a real gain there. `tools/eval/speculative.py` reproduces these measurements.

12. The image encoder

Images reach the model through a separate asset, `instruct_3b.image_encoder` (`model_cls_name ImageTokenizer`). It holds one Neural Engine program with six procedures, one per patch count $N \in \{1152, 1728, 2304, 3600, 4608, 5760\}$, and `swift_preprocess_weights.bin`. The program's inputs are the patches $[N, 3, 14, 14]$, an additive position embedding $[N, 1024]$, RoPE rotation matrices $[N, 32, 2, 2]$ and an additive key mask. Its output is $N/9$ embeddings of width 1536, which the 3B model reads between `<start_of_image>` and `<end_of_image>`. Apple's runtime log names the sizing rule: the aspect ratio is kept, the image is scaled to at most 338 688 pixels (1728 patches), and both sides are snapped to multiples of the 12-patch attention window.

Shape	ViT, 24 layers, $d = 1024$, 16 heads × 64, SwiGLU 2816; 358 M parameters with the merger
Block	$h = \mathrm{rms}(x)g_1$; q, k, v with bias; RoPE then per-head RMSNorm and gain on q, k; attention inside 12×12-patch windows, full attention in layers 7, 15, 23; $x \mathrel{+}= W_o a + b_o$; $x \mathrel{+}= \mathrm{SwiGLU}(\mathrm{rms}(x)g_2)$
Positions	learned 16 × 16 table (bicubic-resized to the patch grid) added after the patch embedding, then a pre-norm; 2D RoPE on interleaved pairs: pair $2f$ rotates by the row, pair $2f{+}1$ by the column, both at $\theta_f = 10000^{-f/16}$
Token order	window-major; inside a window the 3×3 merge groups in raster order; inside a group the patches in raster order
Merger	final RMSNorm; 9 consecutive tokens concatenated (9216) → 4096 → LayerNorm ($\epsilon = 10^{-6}$) → GELU → 2048 → 1536, no biases
Weights	int4 (16-entry LUT × fp16 row scale) except the patch embedding and the last projection (fp16)

Table 7. The image encoder as recovered from its Neural Engine program.

Reading the weights. The int4 blocks resemble the 3B model's, but the header length depends on the convolution's flags: a 16-entry fp16 LUT, 32 zero bytes, the row scales, and the row biases *only if* the convolution has a bias (Neural Engine register 1, bit 0x10). A 128-byte activation table precedes all of this when the convolution applies an activation (bit 0x20000). We first assumed a fixed 128-byte header, which is right only for 16-row blocks with a bias. Every bias-free block (all FFN and merger weights) was then read 4 to 12 inputs out of place, with its last inputs taken from padding. The block sizes look the same either way because each unit is padded to 64 bytes. Compiling known weights with Apple's compiler settled it: a 9216→128 projection, a bias-free up projection and a SiLU-gated projection decode exactly with the flag rule and at chance level (6%) with the fixed header. The effective bias is the stored bias times the row scale, and times 2^{e-32} when the convolution's task writes the Neural Engine bias register `NEBias` with exponent field e (its value field is 0). In this encoder only the V convolutions of layers 18 and 19 do so ($e = 31$); missing it left their biases twice too large, which costs 0.001–0.007 of agreement with Apple's encoder. Projection tiles are written to descending addresses, so channel order is the reversed tile order. The weights are split over two kernel sections; each block is assigned to the section where its LUT is valid. That rule is unambiguous for 1293 of 1297 blocks, and the remaining 4 follow their neighbours. Norm gains are [bias | scale] constant tiles that the planar engine fetches; we take every gain from its fetch offset rather than from a layout formula, because the global-attention layers use a different constant layout. The compiler also folded a 0.01 pre-norm scale into the encoder: the last residual add scales its residual operand by 0.01 (a planar-engine operand scale, the same register that holds 1/8 in the global layers), and 0.1 is folded into each of the last layer's up and down projections, which

are ten times smaller than every other layer's. The converter undoes this. Activations are 33-knot piecewise-linear tables: SiLU on $[-8, 8]$, GELU on $[-4, 4]$, with explicit linear tails.

Ground truth from Apple's own encoder. Unlike the language model, the encoder's MPSGraph executable runs on the Neural Engine from our own process (about 0.8 s per call), so Apple's encoder is a direct oracle for any input we build. A copied package does not load ("Program load failed"), so weights cannot be patched to bisect. We crafted inputs instead. When all N tokens are identical, every attention layer returns V, so the encoder becomes a per-token function of one 1024-vector; and the merger can be inverted from the output. Before the header fix the inversion stalled at cosine 0.991 for every seed; after it, 0.99999. Our NumPy forward then matches Apple's output at cosine 0.987 on constant-token inputs and 0.95–0.97 per token on real images. The llama.cpp port matches Apple's at 0.95 and NumPy at 0.98–0.99, and its Metal and CPU paths agree to 0.9999. Every wiring choice we could vary has a sharp optimum: the alternative V/O and gate/up/down pairings score 0.16–0.23, and in every layer each other choice of V group is worse; on a real image, swapped q/k scores 0.09, half-split RoPE pairs 0.50, x-before-y 0.51 and transposed patches 0.85. Fitting per-layer branch scales gains less than 0.4 points, so the remaining percent is not a wiring error. A Core ML replica of our forward run on the Neural Engine was less faithful still (0.76), so we attribute the gap to the numerics of Apple's compiled program.

Through the 3B model: the RoPE layout. Our prompt places the question, a newline and the image in the user turn, 266 tokens in total, exactly the count Apple's runtime logs for the same request. Several things the encoder program cannot tell us are chosen by Apple's host code: the pixel normalisation, the position table's interpolation, and how the 32 RoPE rotations of a head are split between the row and the column. Comparing against Apple's encoder cannot settle these, because both encoders receive whatever inputs we build. The 3B model can. With the obvious layout (row on pairs 0–15, column on 16–31) the language model received colours and scene gist but no geometry. Shuffling the 192 image tokens hardly changed anything, and asked whether a red circle is left or right, top or bottom, it answered "Left" and "Top" for all four pictures, where Apple's model answers all four correctly. The correct layout *alternates*: pair $2f$ rotates by the row and pair $2f{+}1$ by the column, both at frequency θ_f. With it all four spatial answers are right with margins of 5–12 logits. The mean log-probability of the key words of Apple's captions (circle, HELLO, equation, Golden, balloons, ...) rises from -2.99 to -0.20. The other host choices are then sharp: a negated rotation mirrors the image ("HELLO" at -8.2), bilinear instead of bicubic position interpolation is worse (-0.26), and so is a transposed position table (-0.21). ImageNet normalisation is slightly better than CLIP's (-0.20 vs -0.22). With CLIP's statistics a plain white image is barely "white" (0.2-logit margin over gray, against 6.5 with ImageNet's), and CLIP's mean vector does not occur anywhere in the operating system's shared cache. Teacher-forced on Apple's captions of 13 images (168 tokens), our encoder gives the 3B GGUF Apple's next token 94.0% of the time (NLL 0.29), Apple's own encoder on the same inputs 92.9% (NLL 0.29), and no image 58.9% (NLL 2.11). Free-running through llama.cpp with Apple's instructions, 5 of the 13 captions are identical to Apple's character for character, among them "The word "HELLO" is written in black capital letters on a white background.", and the rest are close paraphrases ("The image shows a math problem: "2 + 3 = ?"." for "... a mathematical equation: 2 + 3 = ?.").

In llama.cpp. `afm2gguf.py --vision` writes the encoder and merger as a multimodal projector (685 MiB, F16), projector type `afm` in llama.cpp's `mtmd` library. Tokens are processed in Apple's order, with a window mask on 21 of the 24 layers. The query and key head dimensions are permuted so that llama.cpp's vision RoPE (rotating dimensions j and $j{+}32$, row for $j < 16$, column above) reproduces Apple's alternating interleaved pairs. Its embeddings match our NumPy forward at cosine 0.990. On an M4 an image encodes in 0.66 s on Metal.

13. The 300M base model

Apple ships a second, smaller language model: `com.apple.fm.language.instruct_300m.base.generic`, display version `afmplus-v11.0-pico`. Fifteen other assets on the same Mac carry the `instruct_300m` name, among them the safety guard (`instruct_300m.safety`), `misc_safety`, `mm_guard`, `action_validator` and several classifiers. In a memory capture of Apple's runtime, the 300M model's input buffer also held a guard turn ending in the model answer `'safe'`. The asset holds one Neural Engine program with 22 procedures (11 with rank-32 adapter slots, 11 with rank-64) and `program.odix` with the embedding.

Shape	24 dense layers, $d = 1024$, 16 query / 4 KV heads × 64 (query head h reads KV head $\lfloor h/4 \rfloor$), SwiGLU 3200; 299 M parameters plus a tied 262 144×1024 embedding
Block	the draft model's layer (Section 11): $h = \mathrm{rms}(x)g_1$; RoPE then per-head RMSNorm and gain on q, k; attention scale 1/8; $x \mathrel{+}= W_o a$; $x \mathrel{+}= \mathrm{SwiGLU}(\mathrm{rms}(x)g_2)$
Positions	RoPE base 500 000, half-split; the cos/sin table ships in `program.odix` directly after the embedding
Embedding	int4 codes × one fp16 scale per dimension; readout $\mathrm{rms}(x)\, g_{\mathrm{out}}\, s$ against the same codes, plus a mask of −40000 on 144 unused ids
Weights	int4: 16-entry fp16 LUT × fp16 row scale; 960 blocks (40 per layer)

Table 8. The 300M base model as recovered from its Neural Engine program.

Reading it. We read it the way we read the draft model and the image encoder: from the Neural Engine task stream rather than from symbol names. Earlier attempts (before this report) decoded every tensor but never produced text, and the reasons turned out to be the same ones the image encoder taught us. (1) The int4 header is 96 bytes (LUT, 32 zero bytes, 16 row scales; the bias flag is clear), followed by the codes and 32 bytes of zero padding; 128-byte headers had shifted every input axis by 4 positions (12 for the 128-row blocks). (2) The norm gains are not folded away: the planar engine fetches all 97 of them (four per layer and a final one) from `__TEXT,__const`, beside a $[0 \mid s]$ tile that scales the hidden state before the readout and the −40000 vocabulary mask. (3) The embedding is a plain 8-token interleave starting at the data section; a lane "skew" and a 4-row shift reported earlier were both produced by a wrong base address. (4) The per-layer multiplier (9.5–12.0) applied with the key gain is not a temperature. It is the int8 scale of the key cache, and the $q{\cdot}k$ matmul's own fp16 scale register divides it back out: their product is 0.1250 in all 24 layers. Values are likewise stored in int8 (scales 305 down to 10.5) and rescaled in the attention-times-V matmul. (5) The two 256-wide projections are emitted in different orders: layer 0 computes K first, every later layer V first, as in the draft model. We assign them by dataflow: K is the one whose output passes through RoPE and the head norm. Block order within a tensor follows the destination addresses (the query blocks and the up projection's are written to descending addresses), as for the encoder.

Results. Apple exposes this model through no public API, so we cannot compare outputs as we do for the 3B. We test it as a language model instead. On two passages of plain English (172 and 102 tokens) its perplexity is 38.2 and 24.6, the median rank of the correct next token is 2, and it ranks it first 29% and 39% of the time. On random token sequences repeated twice it copies the second half 78% of the time, the signature of trained induction heads. Each correction is necessary: with the old K/V assignment induction is 0% and perplexity above 100 000; swapping K and V in layer 0 as well leaves perplexity above 20 000; halving or doubling the logits raises it threefold to ninefold, so the readout is calibrated as decoded. Greedy decoding through llama.cpp:

> The capital of France is → *Paris.* Water boils at → *100°C.*
> Once upon a time, there was a little girl named → *Lily. Lily was a little girl who loved to play with her toys…*

> (chat turn) What is 12 plus 7? → *12 plus 7 is 19.* Write one sentence about the ocean. → *The ocean is a vast, mysterious expanse of water that stretches for miles, teeming with life, diverse ecosystems, and endless horizons.*

In llama.cpp. The layer is the draft model's, so `afm2gguf.py --pico` writes it with the `afm-draft` architecture and no new runtime code: the embedding as Q4_0 with unit scales (exact), the projections as F16, the full 262 144-token vocabulary with Apple's mask as the output bias (721 MiB). Its greedy continuations are identical to our NumPy forward's.

Adapters and the safety guard. Every 300M adapter, including the default one inside the base package (`lora_32/64_constant_data.bin`), is a flat file of fp16 (A, B) pairs, A as $[r, \text{in}]$ and B as $[\text{out}, r]$, per layer in the order attention output, q, k, v, `linear_0`, `linear_1`, FFN output. The one trap is the layer order. The pairs follow the package manifest's tensor names sorted as strings, so the layers come as 0, 10–19, 1, 20–23, 2–9. Read in numeric order, every adapter looks inert (the guard below never says "safe"); with a plain numeric-string sort (0, 1, 10, ...) it half works. `linear_0` is the gate (log-probability of the right first token −0.10 against −0.55 as up), the Neural Engine program scales the product by 0.5, and every role contributes. With the order fixed, the base package's own rank-64 adapter is a trained default: it lowers the base model's perplexity from 38.2 to 30.8 and from 24.6 to 21.4 (llama.cpp, 128-token windows: 26.9 to 22.1). Twelve task adapters of rank 32 or 64 ship on our Mac, and all share its starting point: their weights correlate 0.33–0.99 with the default adapter of the same rank, where unrelated adapters would correlate near 0. The text safety guard (`instruct_300m.safety`, rank 64) carries Apple's instruction: answer `safe`, or one line per problem of the form `category,mention` or `category,action` over 14 categories (hate_harassment, violence_gore, self_harm, jailbreak, vulgarity, ...). Run through llama.cpp with that instruction as the system turn, it answers `safe` for every benign prompt and answer we tried, and labels harmful text in the requested format: `self_harm,action` for a plan to self-harm, `jailbreak,action` for "Ignore all previous instructions...", `vulgarity,action` for swearing. It separates talking about a thing from doing it: a history of the battle of Verdun gets `violence_gore,mention`, a friend's past suicidal thoughts `self_harm,mention`, a news report of an arrest `safe`. The smaller `mm_guard` (rank 32, nine categories) agrees on everything except the swearing, which it calls `safe`; vulgarity is not one of its categories. `afm2gguf.py --pico-adapter NAME` writes any of them as a llama.cpp LoRA with Apple's instructions in its metadata, and `tools/afm300m_classify.py` runs a classifier as Apple renders it. Apple's public FoundationModels API runs the same guard: it rejects a request with "May contain unsafe content" when the guard's labels map to `unsafe` under the API's row of Apple's policy table (`feature_output_category_mapping.json`: eight labels such as `violence_gore,action` or `self_harm,action` block; mentions, jailbreak attempts and vulgarity do not). On 26 prompts (benign questions, mentions, nine harmful requests, two jailbreaks, two vulgar ones) our guard with that table predicts Apple's decision on all 26, including the cases our guard labels but Apple's policy lets through. One request Apple stopped at its output check ("Response may contain sensitive..."), which our guard already flags on the input.

Images. The 300M model has its own image encoder, `instruct_300m.image_tokenizer` (`ImageTokenizer`, model id `afm-plus-v11-300m-instruct-4bit-pal`). Its program has the 3B encoder's structure exactly: 24 layers, windows of 12×12 patches with full attention in layers 7, 15 and 23 (counted from the attention matmuls per layer), the same 3×3 merger, and a last projection to 1024 instead of 1536. Its position table differs, so it was trained separately. Three details differ from the 3B encoder. The weight palettes are sorted real-valued codebooks rather than integers; there is no folded 0.01 scale (the last layer's weights are in line with the others); and 15 of its 96 attention bias tensors come from tasks that write the `NEBias` exponent ($e = 28$–31), so their biases are 2^{e-32} times the stored value. Read without that factor they were 2–16 times too large, which no weight statistic shows but Apple's own encoder does: run on identical inputs it agreed with ours at cosine 0.45–0.54 per token, and after the fix at 0.61–0.78 on photos and 0.97 on constant inputs. The same rule explains every one of the 96 tensors' power-of-two ratio to the 3B encoder's biases.

The same decoder therefore covers both (`afm2gguf.py --pico-vision`); the 3B projector it writes is byte-identical to before. In llama.cpp the encoder runs as the existing `afm` projector. The only runtime change is in the `afm-draft` graph, which now applies the per-dimension embedding scale to text tokens only, so that image embeddings enter unscaled. With the default adapter the 300M model describes images ("The image contains the text "HELLO" in a stylized font."; "a blue square with a white background"). The image classifiers are adapters of the same kind as the guard: `misc_safety` holds Apple's instructions for violence/gore classification and for counting people ("0, 1, 2"), and `prepubescent_safety` checks for children shown with adult features. On stock photos from Keynote's themes the people counter answers 0 for all four photos without people, 1 for all five single portraits (before the bias fix it missed a person whose face is hidden by a hood), 2 for two women, and 2 for three boys, where 2 is the largest answer allowed. The two safety classifiers answer `safe` for every benign image we tried, including synthetic red paint splatter; we did not test them on harmful images. What still separates the encoders from Apple's is shared by both: the last layer writes a massive activation (up to $|x| \approx 6 \cdot 10^4$, at the edge of fp16) into nearly every token, the 3B program pre-scales it by 0.01 and the 300M program does not, and the per-token disagreement concentrates there. Emulating the Neural Engine's softmax (its 33-knot 2^x and $2/m$ tables and fp16 $\log_2 e$) and fp16 saturation changes nothing.

14. Fidelity

The reference is Apple's own model, queried through the public FoundationModels API at temperature 0 with the instructions "You are a helpful, concise assistant." (`tools/afm.swift`) on 40 prompts. We measure two things: *teacher-forced* top-1 agreement (Apple's answer is fed in and we count how often our most likely next token is Apple's; 1127 answer tokens) and *free-running* agreement (our own greedy answer compared with Apple's, character for character).

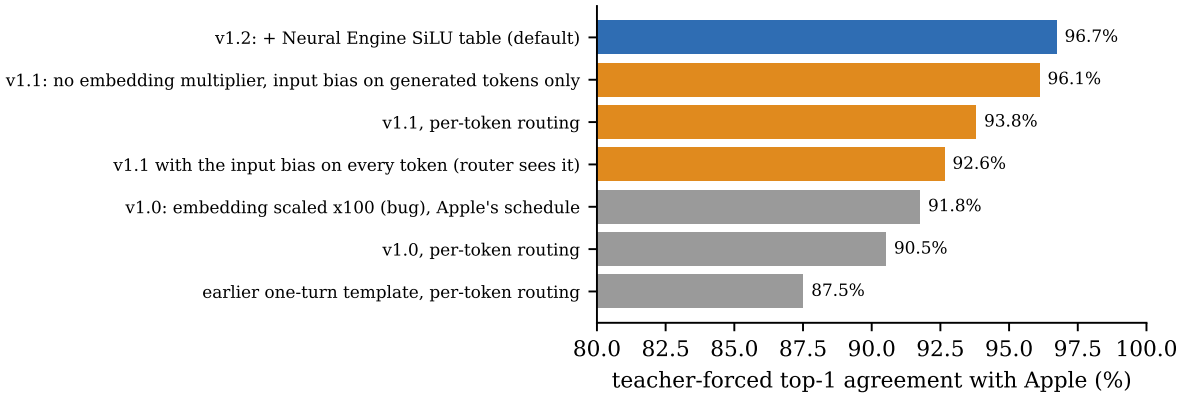


Figure 11. Teacher-forced top-1 agreement with Apple's greedy answers (40 prompts, 1127 tokens). Blue: the shipped default. The template fix is worth three points, the layer-0 fixes (no embedding multiplier, input bias on generated tokens only) four more, and Apple's routing schedule beats per-token routing.

The layer-0 input: no multiplier, and a bias only for generated tokens. Earlier drafts fitted an input multiplier k and found the score flat for $k = 8$–14. That scan was invalid: llama.cpp's generic embedding code already applied the GGUF's `embedding_scale`, and the AFM graph applied it again, so the scan really covered $k = 80$–140. Dumping llama.cpp's intermediate tensors and comparing them with an independent NumPy forward exposed a factor of exactly 10 at the layer-0 input. Scanning the true multiplier with the expert picks held fixed gives a sharp optimum at $k = 1$

Routing (exact template)	teacher-forced	first token	identical answers
v1.3 default (+ no bias at re-pick tokens)	**96.98%**	38/40	17/40
Apple's schedule + Neural Engine SiLU	96.72%	38/40	17/40
Apple's schedule, exact SiLU	96.10%	37/40	17/40
per token	93.79%	36/40	12/40
v1.0: Apple's schedule, embedding ×100, bias on every token	91.75%	35/40	7/40
v1.0: per token	90.51%	38/40	11/40

Table 9. Routing rules. "Identical answers" = our free-running greedy answer equals Apple's exactly (up to 200 tokens); one different early token sends the rest of an answer elsewhere, so this measure is dominated by the first few tokens.

Ablation (earlier 10-prompt set, earlier template)	top-1
always-on + 23 routed experts	85.6%
always-on experts only	62.9%
routed experts only	1.4%
16 instead of 23 routed / 14 routed	83.5% / 82.1%
standard (pre-norm) residual instead of x/y	0%

Table 10. Ablations: both expert groups are needed, the routed count matters, and the residual form is not negotiable.

(mean log-loss of Apple's tokens 0.140, against 0.143 at 0.75 and 1.25 and 0.240 at 100). This matches what Apple's files say: the gather has no multiply.

With $k = 1$ the body of the network and the router disagreed. The body wanted the input bias added, but the router's picks were best when the bias was negligible. We found this by computing the picks from a forward at a larger k, or with the bias removed; a version that changed only the layer-0 attention balance did not help (78%). Apple's ANE program has two procedure families. The prompt procedures (`prompt_opt`) add a zero constant at layer 0. The extend procedures, which compute generated tokens, add the bias vector. Applying the bias to generated tokens only is one consistent model for both parts. It reaches 95.9% with the picks held fixed and 96.1% in the shipped runtime, which gives the bias to single-token decode steps and not to prompt chunks. Flipping the bias's sign costs two points, so the vector is real; it simply belongs to one procedure family.

The tokenizer. Apple's `SystemLanguageModel.tokenCount` gives Apple's own token count. Our first tokenizer (and the research encoder) matched it on 73 of 81 texts; it had byte-fallback tokens labelled as normal text and control tokens mislabelled. The converter's tokenizer matches on 81/81 (and on 16/16 texts built from tabs, runs of spaces and indented code) and lifted fidelity from 86.0% to 87.5%. Apple's tokenizer file marks all user-defined pieces with one type code; mapped naively, llama.cpp prints space runs as the escape character U+2581 instead of spaces and drops tab runs entirely, so the converter types space runs as normal text and tab runs as user-defined.

15. The converter

`afm2gguf.py` needs only NumPy and gguf-py and contains no Apple data. It finds assets by their `AssetSpecifier`, decodes everything directly, and refuses to write if a check fails: palette and container layout, 3504 units in 219 chunk-sets, the router mask, the tokenizer layout, the embedding scale array (chosen by row-norm consistency) and—decisively—the gain-table alignment, chosen by the FFN coupling-sign agreement (0.99 for the true start, 0.59 for the one-vector-early candidate that passes every shape test). Its output is byte-identical, tensor by tensor, to the validated export. With

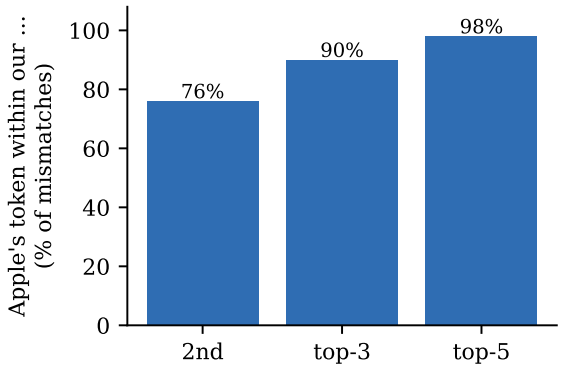


Figure 12. Where the 93 disagreements of the v1.0 configuration sat: Apple's token is our second choice in 76%, in our top three in 90% and in our top five in 98%; the median logit gap to it is 1.15, and 45% of the misses are within one logit.

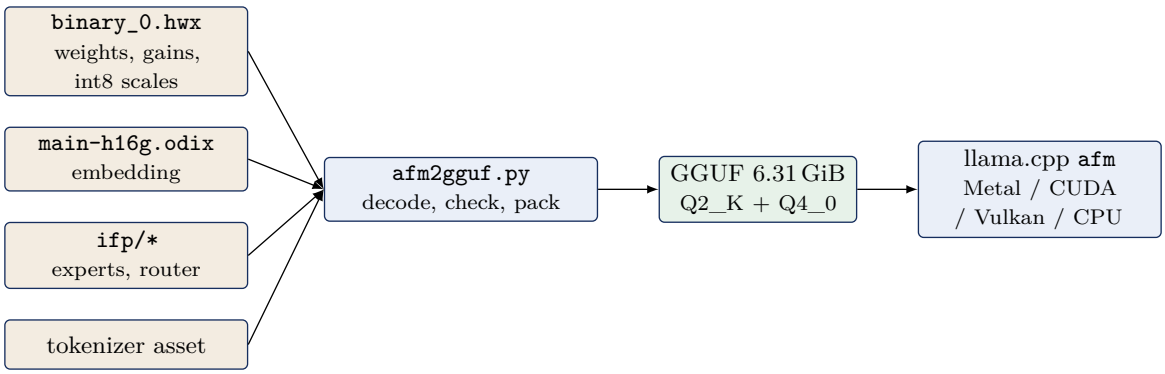


Figure 13. Conversion runs on the Mac that holds the assets (2–3 minutes); the GGUF then runs anywhere.

`--draft`, `--vision`, `--pico`, `--pico-vision` and `--pico-adapter` it writes the draft model, the image encoder, the 300M base model, its image encoder and its adapters instead; the image encoder's weights are bit-identical to the research decoder's, with every gain taken from the program's own constant fetches, and the 300M model's RoPE base is fitted from the table in its own asset (500 000).

16. What did not work

Recorded so that nobody repeats them:

- Running Apple's backbone on the ANE from our own process: the kernel refuses LLM base-model requests from non-platform binaries, even with AMFI disabled.
- Ten-plus forward variants before the gain fix: Gemma-style $1+g$ gains, standard residual, swapped or negated RoPE tables, embedding scales, alternative row stripes, GQA maps, within-vector gain permutations, per-row scale permutations—all 0% top-1, because the defect was elsewhere.
- LoRA as the missing piece: the Foundation Models API's generic adapter for v11.2 is untrained (B RMS $3 \cdot 10^{-5}$), so the base model is already what the public API runs. Its "signature" (the SHA-256 of the empty string) proves nothing, because the trained feature adapters carry the same one (Section 10).
- Apple's `MODLM current context` N `asked to run` K `tokens` log as an oracle for draft acceptance: it interleaves the guard model, the target and the draft (and several generations)

with no stream id, and the draft's catch-up calls are only ever 1 or 3 tokens, so per-step acceptance cannot be read off it.

- Patching Apple's image encoder to bisect it: a copied MPSGraph package does not load on the Neural Engine. A Core ML replica of our encoder on the Neural Engine was less faithful (0.76) than NumPy (0.987).
- Before the alternating RoPE layout was found, we tried to restore spatial understanding with RoPE position rescaling (the RoPE table has 481 rows; $\times 10$ or fit-to-480 positions), window-relative positions, other window, group, patch and output orders, cropped position tables, collapsed LLM positions for the image tokens, bidirectional attention over them, rescaled embeddings, no int8 K/V emulation and image-before-question prompts. None helped. Each of them leaves the axis layout unchanged, so they could not.
- Weight statistics such as "alignment ratios" and residual-collapse measures: uncalibrated, and on a healthy model indistinguishable from AFM.
- The earlier attempts on the 300M model (from symbol names, compiled probes and permutation searches) recovered the tile element order exactly but never produced text. Their conclusions—no norm gains ship, the embedding has a lane skew and a 4-row shift, the residual-writing projections hide an unreadable layout, some layers' FFNs "hurt"—were all consequences of the 128-byte header, the wrong embedding base and the positional K/V assignment (Section 13).
- Reading the 300M adapters' layers in numeric order. Every adapter then looks inert, and the base package's default adapter even looks slightly harmful; we briefly took it for a placeholder. The files follow the manifest's string-sorted tensor names (Section 13).
- Pooled expert routing rules (mean of inputs, of logits, of probabilities; prompt or user turn; prefix mean, vote counts): all below per-token routing, and all wrong—Apple routes from a single token (Section 8). They had looked plausible only because our template was wrong.
- Routing from the layer-12 FFN input instead of its attention input: 89.4% teacher-forced, and three times too many experts changed per re-pick (109 versus Apple's 36).
- Reading Apple's between-prompt expert changes from the cache log: those counts (361–556 of 1012) include cache eviction while the service is idle and do not measure pick differences; only re-picks within one generation do.

17. Toward 100%

The prompt template, the expert schedule, the layer-0 input and the activation function are now Apple's own. 37 of 1127 answer tokens still differ. Apple's token is almost always our second or third choice. Earlier drafts called the remaining gap a numerical noise floor; the layer-0 bug shows that this was premature. What we measured directly:

(1) **The ANE's matrix multiplies are accurate.** We ran our own CoreML programs on the Neural Engine, confirming the device with Core ML's compute plan. Sums accumulate at high precision and only the fp16 output is rounded (relative error 2.8×10^{-4}, the same as fp32 accumulation with an fp16 result). The low-precision accumulation one first sees comes from Core ML's fp16 CPU path, not the ANE.

(2) **Its SiLU is a 33-knot piecewise-linear table** (knots every 0.5 on $[-8, 8]$, fitted values, for example 0.2964 at $x = 0.5$ where SiLU is 0.3112). The same 128-byte table appears 2113 times in AFM's ANE program, once per FFN task, and it is the program's only activation table. We tabulated it for all 65 536 fp16 inputs. It is exactly linear interpolation between the knots, with a left tail clamped to zero below -10.2 and $x - 0.0039$ above 8, rounded to fp16. In the v1.0 setup emulating it was neutral (10 fixed, 8 broken) because the layer-0 error dominated. On the corrected model it fixes 14 and breaks 5 tokens (96.10% $\rightarrow$ 96.72%),

so it is now the default. Its softmax, exp and rsqrt are also approximate (0.3–0.4% relative error).

(3) **Apple's speculative decoding is exact.** Every FM-API adapter sets `draft_soft_match_tolerance` to 0 and turns rejection sampling off, so the draft model never changes a greedy token. The FM-API adapter is the declared-empty adapter (signature = SHA-256 of the empty string), and there is no repetition penalty.

(4) **The router is robust to arithmetic.** Computing it with fp16 accumulation moves 0.01 of 23 experts per layer-pick, and random noise strong enough to swap a third of an expert per layer changes 2% of predictions without moving the agreement.

Every weight, gain, bias, the router, the tokenizer, the template and the routing rule come from Apple's files and code; nothing was trained or fine-tuned.

Path noise. On the final model, the CPU and Metal backends (identical mathematics) disagree on 0.71% of answer tokens (8/1127), and their agreement with Apple is identical (96.72%). The remaining 37 misses are near-ties: Apple's token is our second choice in 84% and always in our top three, and the median logit gap is 0.40. Rounding every matmul output to fp16, as the ANE stores it, gives 96.89% (within noise). In v1.0, mathematically identical variants disagreed on 0.1–2.4% of next-token choices: fp16-rounded activations against fp32, llama.cpp's RoPE kernel against an elementwise one, or one batch against a prompt batch followed by single steps. The 2-bit weights and the int8 KV cache amplify rounding, so a percent or so of residual disagreement is expected even from an exact model.

A larger test. The 40-prompt set moves by about ±0.4% from noise alone, so we added 190 prompts (6572 answer tokens in all) and re-ran every candidate (Table 11). The Neural Engine SiLU is worth a full point. Every other ANE detail we could emulate is neutral: fp16 matmul outputs, int8 KV on or off, keys quantised before their gain, and fp16 RoPE tables. Truncating instead of rounding the int8 cache costs a point, so Apple rounds to nearest. Of the remaining 316 misses, 2.9% of tokens are missed under any routing and 1.9% only under Apple's schedule. The latter is consistent with our prompt-level pick differing from Apple's by a boundary expert or two: the 23rd and 24th router logits are a median 0.003 apart, about the size of a rounding difference in the layer-12 state. Apple's log also confirms the re-pick timing. Around a boundary the runtime steps one token at a time, and the pick at position 96 uses token 96 itself. Apple's fp16 logits never tie at the top, and its softmax errors (about 0.2% in the denominator) cancel in the post-attention RMS norm.

Variant (230 prompts, 6572 answer tokens)	top-1	fixed / broken
v1.3 default (no input bias at re-pick tokens)	**95.27%**	12 / 7
v1.2 default (Apple's schedule, ANE SiLU, int8 KV)	95.19%	–
exact SiLU	94.17%	72 / 139
per-token routing	93.34%	–
fp16 rounding of every matmul and attention output	95.13%	30 / 34
int8 KV emulation off	95.07%	46 / 54
int8 rounding by truncation	94.05%	64 / 139
keys quantised before their gain	95.09%	43 / 50
RoPE cos/sin rounded to fp16	94.99%	21 / 34
re-picks that favour already-loaded experts ($\beta = 5 \cdot 10^{-4}$)	95.15%	–

Table 11. Candidates on the larger set, against the v1.2 default (fixed / broken = tokens that become right / wrong).

Running the dense layers on the Neural Engine. As a direct test of the arithmetic, we rebuilt layers 0–11 as a Core ML program, which placed every operation on the ANE: int8 KV round and clip, the ANE's own SiLU and softmax. We fed its layer-12 state to llama.cpp for routing and the remaining

44 layers. The state differs from our emulation by about 15% (relative norm) after 12 layers; in isolation each layer's FFN agrees with our SiLU-table model to fp16 rounding (0.5%), and int8 steps in the very coarse layer-0 values amplify the differences. Agreement with Apple stays the same: 94.92% against 95.19%, but 224 of 6572 tokens flip (103 fixed, 121 broken). The control, the same program in fp32 on the CPU with exact SiLU, gives 94.42%. So two plausible versions of the hardware arithmetic disagree on about 3.4% of tokens without either being closer to Apple, which is the size of the remaining gap. Core ML compiles our graph differently from Apple's hand-built ANE program, so even the real hardware does not reproduce Apple's rounding. Closing the last points would need that program's exact arithmetic.

Apple's own expert decisions. Apple's runtime logs every pick per layer ("layer N experts: ..."). The expert list is marked *sensitive*, so it stays redacted even with a logging profile that enables private data for the inference service. That profile does reveal the subsystem's ordinary private values. The cache statistics are public, though. At each pick the runtime logs `reordered`, `replaced` and `loaded`, and within one answer the cache holds exactly the current pick, so `replaced` at a re-pick counts the experts that changed. Over the 135 re-picks in the 230-prompt set, our router's predicted churn tracks Apple's with $r = 0.934$ and a mean miss of 4.6 of about 40 changed experts. That bounds our picks to within a few of Apple's 1012 per pick. Moving the re-pick to a neighbouring token makes the match clearly worse (5.3–5.4), which pins the schedule. Every arithmetic variant scores the same within noise: 4.50 for the ANE-computed layer-12 state, 4.60 for the default, 4.60 for fp16 matmul outputs, 4.65 for fp32 with exact SiLU, 4.67 for exact SiLU, and 4.45 without the input bias at re-pick tokens. Apple's re-picks change slightly fewer experts than ours (by 1.6 on average). A small preference for already-loaded experts fits the counts only marginally better (4.60 to 4.40) and costs words (95.15%), so we reject it. Apple's code runs a dense-only function at each re-pick token, the same procedure family as the prompt, which adds no layer-0 bias. Dropping the bias at those tokens is v1.3's only change: 95.19% → 95.27% (12 fixed, 7 broken). Between prompts the counts do not track the previous prompt's pick (Apple 501 on average, our prediction 111), so the cache is not simply carried over between requests.

What the misses were in v1.0. With Apple's schedule, 74 of the 93 v1.0 misses were also missed with per-token routing, so they did not depend on routing. At 89 of the 93 our choice was an ordinary text token, so Apple's output masking is not involved. Table 12 lists the numerical experiments, run on the v1.0 configuration.

Variant of the default (teacher-forced, 1127 tokens)	top-1
v1.0 default: Apple's schedule, int8 KV emulation, embedding ×100	91.75%
int8 KV emulation off	90.77%
no re-pick during generation	90.95%
re-pick takes effect one token later	91.39%
activations rounded to fp16 (residual / + router input / + every sublayer output)	91.39%
RoPE angles computed in fp16 / + fp16 cos and sin	91.75% / 91.57%
attention scale ×0.8 / ×0.9 / ×1.1 / ×1.25 (of $1/\sqrt{128}$)	87.8 / 90.1 / 89.9 / 89.9%
tanh soft-cap of attention scores at 50 / 30 / 20 (Gemma-style)	91.4 / 90.4 / 89.5%
our own fp32 RoPE instead of llama.cpp's (same mathematics)	91.84%

Table 12. Numerical and schedule experiments. Emulating Apple's int8 KV cache and re-picking every 32 tokens are both worth close to a point, the attention scale peaks sharply at $1/\sqrt{128}$, and there is no attention soft-cap; storing activations or RoPE angles in fp16 changes nothing beyond the noise floor.

18. Responsible use

The model's weights belong to Apple. We publish only our own code (the llama.cpp architecture patch and the converter, at `https://github.com/yu314-coder/afm-ifp-teardown`), which rebuilds the model from the assets already on a user's Mac; we publish no weights, tokenizer data or decoded assets, and have asked Apple whether redistribution would be permitted. The project is unofficial and not affiliated with or endorsed by Apple.

Appendix A. GGUF layout

Tensor	Shape (rows × cols)	Type
`token_embd`	262144×1536	Q4_0
`input_bias`, `output_norm`, `output_norm_2`	1536	F32
`blk.L.attn_q`	2048×1536	Q2_K
`blk.L.attn_k`, `attn_v` ($L \leq 34$)	1024×1536	Q2_K
`blk.L.attn_output`	1536×2048	Q2_K
`blk.L.attn_q_norm`, `attn_k_norm` ($L < 34$)	128	F32
`blk.L.{post_attention,ffn,post_ffw,layer_output}_norm`	1536	F32
`blk.L.ffn_{gate,up}` / `ffn_down` ($L < 12$)	3072×1536 / 1536×3072	Q2_K
`blk.L.ffn_{gate,up}_shexp` / `ffn_down_shexp` ($L \geq 12$)	5888×1536 / 1536×5888	Q2_K
`blk.L.ffn_{gate,up}_exps` / `ffn_down_exps`	$348 \times 256 \times 1536$ / $348 \times 1536 \times 256$	Q2_K
`blk.12.ifp_router_fc1` (+bias), `_norm`, `_fc2` (+bias)	256×1536, 256, 15312×256	F32

Table 13. 805 tensors, 6.31 GiB. Metadata keys under `afm.*`: block count 56, embedding length 1536, head counts 16/8, key length 128, expert count 348, experts used 23, shared expert width 5888, leading dense blocks 12, KV-reuse first layer 35, RoPE base 5×10^5, RMS ε, embedding scale, per-layer K/V int8 scales, and a Jinja chat template.

Email address: `euler.yu@gmail.com`